

北京理工大学

人工智能基础项目报告

基于 PyTorch 的小型 GPT 模型构建与训练

Building and Training a Small GPT Model with PyTorch

学 院： 自动化学院

专 业： 自动化

学生姓名： 姜尹雯

学 号： 1120233551

指导教师： 周志强

2026 年 6 月 7 日

目 录

第 1 章	绪论	1
1.1	项目背景与意义	1
1.2	项目任务与技术路线	1
第 2 章	大语言模型相关理论基础与关键知识点	2
2.1	GPT 类语言模型基本原理	2
2.2	文本分词与输入表示	3
2.3	Transformer 核心机制	4
2.4	预训练与微调机制	5
第 3 章	小型 GPT 模型设计与实现	6
3.1	模型总体结构	6
3.2	核心模块设计	6
3.2.1	MiniGPTConfig 配置类	6
3.2.2	因果多头自注意力模块	7
3.2.3	前馈网络与 Transformer Block	7
3.2.4	MiniGPT 模型主体	8
3.3	模型参数配置与低算力适配	8
第 4 章	微调实验与任务适应	9
4.1	微调任务与监督数据构建	9
4.2	微调策略与程序设计	11
4.3	微调结果与任务效果分析	12
4.4	本章小结	15
参考文献与开源资源说明		16
附录		16
附录 A: 项目文件结构		16
附录 B: 主要运行命令		16
附录 C: 核心代码说明		16
附录 D: 实验结果截图清单		16

第 1 章 绪论

1.1 项目背景与意义

近年来，以 GPT 为代表的大语言模型在自然语言处理、智能问答、文本生成、代码生成等领域得到了广泛应用。大语言模型通常基于 Transformer 结构，通过大规模文本数据训练，使模型能够学习语言中的上下文关系和文本生成规律。与传统自然语言处理方法相比，大语言模型具有更强的通用性和迁移能力，已经成为当前人工智能领域的重要研究方向之一。

本项目以“从零构建大模型”为主题，重点并不是训练一个真正具有大规模参数量和复杂能力的商用模型，而是在低算力平台上实现一个小参数量 GPT 类语言模型。通过完成模型搭建、文本数据处理、预训练、微调和推理展示等过程，可以更加直观地理解大语言模型的基本结构和工作原理，从实践角度加深对 Transformer、注意力机制、语言建模以及微调方法的认识。

1.2 项目任务与技术路线

根据课程作业要求，本项目需要围绕“从零构建大模型”的完整流程展开。首先，需要梳理大语言模型构建过程中涉及的重要知识点，包括 GPT 类模型的基本架构、模型参数配置、文本数据准备、分词方法、词嵌入、位置编码、预训练目标以及微调方法等内容。其次，需要基于 PyTorch 搭建一个小参数量 GPT 类语言模型，使其能够在普通单机或较低算力平台上完成训练和推理，并体现大语言模型中的核心机制。然后，需要准备小规模文本语料，完成文本编码、训练样本构造和预训练程序设计，使模型能够通过预测下一个 token 的方式学习基本语言规律。在此基础上，还需要构建监督微调数据，选择合适的微调任务和训练策略，使预训练后的模型能够适应特定任务。最后，需要对实验结果进行展示和分析，总结项目完成情况、遇到的

问题以及学习收获。

本项目的整体技术路线可以概括为以下几个步骤：

1. 数据准备与分词编码：选取 Hugging Face 平台上的 TinyStories 数据集作为预训练语料，对文本进行清洗和拼接，并采用基于正则表达式的 tokenizer 将文本转换为 token id 序列；
2. 训练样本构造：利用滑动窗口方法将 token 序列切分为固定长度的输入序列，并将其右移一位作为预测目标，构成 next-token prediction 训练样本；
3. 小型 GPT 模型搭建：基于 PyTorch 构建小参数量 GPT 类模型，主要包括 token embedding、position embedding、因果多头自注意力、前馈网络、残差连接和层归一化等模块；
4. 预训练与生成测试：使用 TinyStories 样本对模型进行预训练，通过交叉熵损失优化下一个 token 的预测结果，并在训练后根据给定 prompt 进行自回归文本生成；
5. 微调与结果分析：在预训练模型基础上构建简单监督任务进行微调，并结合 loss 曲线、perplexity、生成样例和微调结果，对模型效果进行分析总结。

第 2 章 大语言模型相关理论基础与关键知识点

2.1 GPT 类语言模型基本原理

自回归建模思想。 GPT 类语言模型的核心任务是根据已有上下文预测下一个 token。文本经过分词后可表示为 token 序列 $x_1, x_2, \dots, x_T$ ，模型并不是一次性生成完整文本，而是按照从左到右的顺序逐步建模当前位置的条件概率。对应到训练阶段，next-token prediction 要求模型根据前 t 个 token 预测第 $t + 1$ 个 token，其语言建模目

标可写为：

$$P(x_1, x_2, \dots, x_T) = \prod_{t=1}^T P(x_t | x_{<t}),$$

$$\mathcal{L}_{\text{LM}} = -\frac{1}{T-1} \sum_{t=1}^{T-1} \log P(x_{t+1} | x_{\leq t}).$$
(2-1)

式中，第一行表示自回归概率分解，第二行表示下一个 token 预测任务中的平均交叉熵损失。

Decoder-only 结构。为了服务于从左到右的生成目标，GPT 采用 Decoder-only Transformer 结构，并通过因果遮罩限制每个位置只能使用当前位置及其之前的信息。这样，模型在训练时不会提前看到未来 token，在推理时也可以不断把新生成的 token 拼接回上下文，继续预测后续内容。本文实现的 MiniGPT 是该结构的小型化版本，保留 token embedding、position embedding、因果自注意力、前馈网络、残差连接和 LayerNorm 等关键组件，但通过较小的层数、嵌入维度和上下文长度降低训练成本。图 2-1 给出了大语言模型从文本输入到模型输出的整体结构关系，可作为后续 MiniGPT 实现流程的概念参考。

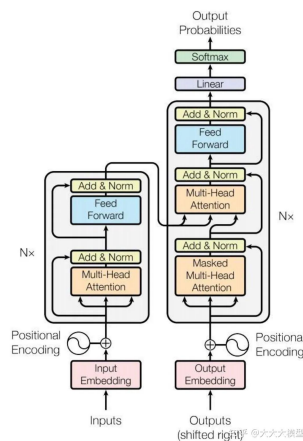


图 2-1 大语言模型整体结构示意图

2.2 文本分词与输入表示

Tokenizer 的作用。神经网络不能直接处理原始字符串，因此文本需要先经过 tokenizer 转换为 token，再映射为整数形式的 token id。Tokenizer 主要完成文本切分、

词表构建、编码和解码：编码将文本转为模型输入，解码则将模型生成的 token id 还原为可读文本。本项目保留正则 tokenizer 和 GPT-2 BPE tokenizer 两种方案。正则 tokenizer 词表较小，更适合小模型和低算力训练；GPT-2 BPE tokenizer 更接近真实 GPT 系列模型常用的子词分词方式，但固定词表较大，会增加输出层参数量。预训练语料中使用 `<|endoftext|>` 作为故事边界标记，具体数据拼接、缓存和样本构造过程放在第四章说明。

输入表示。得到 token id 后，模型还需要将离散编号映射为连续向量。GPT 的输入表示由 token embedding 和 position embedding 相加得到：

$$\mathbf{h}_0 = \mathbf{E}_{\text{tok}}(\mathbf{x}) + \mathbf{E}_{\text{pos}}(\mathbf{p}). \quad (2-2)$$

其中， $\mathbf{x}$ 表示 token id 序列， $\mathbf{p}$ 表示位置编号序列。Token embedding 提供 token 本身的表示，position embedding 提供顺序信息，二者共同构成 Transformer Block 的输入。

2.3 Transformer 核心机制

Transformer Block 的作用。GPT 的主体由多层 Transformer Block 堆叠而成。Transformer Block 通过因果自注意力建模上下文关系，再通过前馈网络进行非线性变换，并使用残差连接和 LayerNorm 稳定训练。对本文项目而言，需要重点理解的是：自注意力让模型能够从上下文中选择相关信息，因果遮罩保证模型不能看到未来 token，多层堆叠则逐步形成更高层次的文本表示。图 2-2 展示了 Transformer 模块中注意力、前馈网络以及残差归一化结构之间的关系。

因果自注意力机制。注意力计算通常由 Query、Key 和 Value 三组向量完成。在 GPT 中，为了满足自回归生成约束，需要在注意力分数中加入因果遮罩矩阵 $\mathbf{M}$ ：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T + \mathbf{M}}{\sqrt{d_k}} \right) \mathbf{V}. \quad (2-3)$$

其中， $\mathbf{M}$ 表示因果遮罩矩阵，未来位置对应的注意力分数被置为负无穷，从而避免

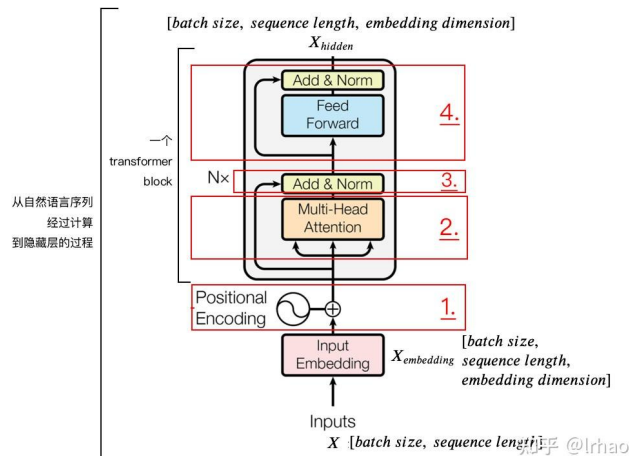


图 2-2 Transformer 核心结构示意图

模型看到未来 token。多头注意力则是在多个子空间中并行执行上述注意力计算，使模型能够从不同角度捕捉上下文关系。

2.4 预训练与微调机制

预训练。预训练使用无标签文本作为数据来源，通过 next-token prediction 让模型学习通用语言模式。本项目的预训练阶段对应 TinyStories 语料，目标是让小型 MiniGPT 在有限数据和算力条件下学习基本英文短故事中的局部上下文关系。预训练结果主要通过训练损失、验证损失、perplexity 以及生成样例进行观察，具体实验设置和结果放在第四章展开。

微调。微调是在预训练模型基础上，使用带标签或具有明确任务格式的数据继续训练，使模型适应具体任务。与预训练关注通用语言规律不同，微调更关注任务目标，例如文本分类或问答。本文后续微调实验会在预训练 MiniGPT 的基础上构建简单监督任务，相关分类头设计、损失函数和评价指标放在第五章说明。

第 3 章 小型 GPT 模型设计与实现

3.1 模型总体结构

本项目实现的是一个小参数量 Decoder-only GPT 类语言模型，其目标是在普通单机和较低算力条件下完成模型结构验证、预训练和推理生成实验。模型输入为 token id 序列，形状为 $[\text{batch_size}, \text{seq_len}]$ ；模型输出为 logits，形状为 $[\text{batch_size}, \text{seq_len}, \text{vocab_size}]$ 。其中，最后一维对应词表中每个 token 的预测分数，可用于预训练阶段的 next-token prediction 损失计算，也可用于推理阶段选择下一个 token。

从整体结构看，MiniGPT 包含 token embedding、position embedding、embedding Dropout、多层 TransformerBlock、final LayerNorm 以及无偏置线性输出层。该结构与第二章介绍的 GPT 基本原理一致，但参数规模明显减小，更适合课程实验中对大语言模型核心机制进行复现和验证。

输入 token ids $\rightarrow$ Token Embedding + Position Embedding $\rightarrow$ Transformer
Block $\times N \rightarrow$ Final LayerNorm $\rightarrow$ Linear Head $\rightarrow$ logits

上述流程体现了小型 GPT 的基本计算路径：离散 token id 首先被映射为连续向量表示，再经过多层因果 Transformer Block 建模上下文关系，最终输出词表维度上的 logits。后续预训练、文本生成和微调实验均建立在这一模型主体之上。

3.2 核心模块设计

3.2.1 MiniGPTConfig 配置类

MiniGPTConfig 用于集中管理模型结构参数，包括 vocab_size、context_length、emb_dim、n_heads、n_layers、dropout 和 qkv_bias 等。这些参数分别决定词表规模、最大上下文长度、嵌入维度、注意力头数、Transformer Block 层数、正则化强度以及 Query、Key、Value 线性映射是否使用偏置。

采用配置类的好处是便于在不同实验中快速调整模型规模。例如，当训练平台算力有限时，可以减小 `emb_dim`、`n_layers` 或 `context_length` 来降低显存占用和训练时间；当数据规模扩大时，也可以适当增加模型容量。该设计使模型结构和实验参数之间保持清晰对应，便于后续预训练实验记录和报告分析。

3.2.2 因果多头自注意力模块

`CausalSelfAttention` 是 MiniGPT 中最能体现 GPT 自回归建模机制的模块。输入隐藏状态首先分别经过线性层映射为 Query、Key 和 Value，然后将嵌入维度按照注意力头数划分为多个子空间，在每个头中独立计算缩放点积注意力。多头注意力的结果重新拼接后，再经过输出投影层得到与输入维度一致的表示。

为了保证模型只能根据历史信息预测下一个 token，代码中构造了上三角因果 mask，用于屏蔽当前位置之后的未来 token。该 mask 通过 `register_buffer` 注册为 buffer，因此会随模型一起移动到 CPU 或 GPU，但不会作为可训练参数更新。因果注意力的核心计算可概括为：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T + M}{\sqrt{d_k}}\right)V, \quad (3-1)$$

其中， M 表示因果遮罩矩阵。通过该机制，模型在训练时无法利用未来 token，从而与 next-token prediction 的任务目标保持一致。

3.2.3 前馈网络与 Transformer Block

`FeedForward` 模块采用“两层线性层 + GELU + Dropout”的结构。第一层线性层将隐藏维度从 `emb_dim` 扩展到 $4 * \text{emb_dim}$ ，经过 GELU 非线性激活后，再由第二层线性层映射回 `emb_dim`。这种先升维再降维的设计是 GPT 类 Transformer Block 中的常见结构，用于增强每个位置隐藏表示的非线性表达能力。

`TransformerBlock` 由因果自注意力子层和前馈网络子层组成，并采用 Pre-LayerNorm 形式。具体来说，每个子层之前先进行 LayerNorm，然后通过对应模块计算输出，

最后与原输入进行残差相加。残差连接有助于信息和梯度跨层传播，LayerNorm 能够稳定训练过程，Dropout 则用于降低小数据训练时的过拟合风险。通过堆叠多个 TransformerBlock，MiniGPT 能够逐层建立 token 之间的上下文依赖关系。

3.2.4 MiniGPT 模型主体

MiniGPT 是整个模型的主体类，内部定义了 token embedding、position embedding、embedding Dropout、多个 TransformerBlock、final LayerNorm 和无偏置线性输出层。在 forward 过程中，模型首先检查输入序列长度是否超过 context_length；随后根据当前序列长度自动生成 position id，并分别计算 token embedding 和 position embedding。二者相加后经过 Dropout，再依次输入 Transformer blocks、final LayerNorm 和线性输出层。

最终得到的 logits 可直接用于预训练阶段的交叉熵损失计算。推理生成时，程序则取最后一个位置的 logits，根据贪心策略或采样策略选择下一个 token，并将其追加到上下文中继续生成。因此，MiniGPT 模型主体同时服务于预训练、文本生成和后续微调实验。

3.3 模型参数配置与低算力适配

本项目并不追求训练真实大规模语言模型，而是构建一个能够体现 GPT 核心机制的小参数量 MiniGPT。为了适应单机低算力平台，实验中通过较小的 emb_dim、n_heads、n_layers、context_length 和 batch size 控制训练成本。同时，qkv_bias 默认为 False，输出层采用无偏置线性层，也有助于减少参数量。Dropout 被用于降低小数据训练中的过拟合风险。

表 3-1 中的参数来自 TinyStories 5k 预训练实验记录。可以看出，模型通过降低嵌入维度和 Transformer 层数，将可训练参数量控制在约一百二十万级别。相比真实大语言模型，该规模非常小，但已经足以展示 GPT 类模型中的 token embedding、位

表 3-1 小型 GPT 模型主要参数配置

参数名称	含义	设置值
vocab_size	词表大小	8785
context_length	最大上下文长度	64
emb_dim	词嵌入维度	64
n_heads	注意力头数	4
n_layers	Transformer Block 层数	2
dropout	Dropout 比例	0.2
qkv_bias	QKV 映射是否使用偏置	False
参数量	可训练参数量	1,228,288

置嵌入、因果多头自注意力、前馈网络、残差连接和 LayerNorm 等关键机制。

代码中还提供了 `count_parameters()` 函数,用于统计模型可训练参数量,方便将不同实验配置下的模型规模写入日志和报告。`simple_test()` 则使用随机 token ids 测试模型输入输出形状,确认输入 `[batch_size, seq_len]` 后能够得到 `[batch_size, seq_len, vocab_size]` 的 logits,从而验证模型结构搭建正确。

综上,本章完成了小型 GPT 模型架构的搭建。该模型保留了 token embedding、position embedding、因果多头自注意力、前馈网络、残差连接和 LayerNorm 等核心机制,同时通过较小参数规模适配单机低算力环境。该部分直接对应课程作业中“小参数量 GPT 类大语言模型架构搭建”的要求,并为后续预训练、推理生成和微调实验提供模型基础。

第 4 章 微调实验与任务适应

4.1 微调任务与监督数据构建

预训练阶段的 MiniGPT 主要学习根据上下文预测下一个 token,这一目标能够让模型掌握一定的英语短故事语言模式,但尚不能直接完成明确的下游分类任务。为了考察预训练模型的任务适应能力,本文在预训练 MiniGPT 的基础上构建故事结局

倾向二分类任务（`story ending polarity classification`），要求模型根据输入的故事结尾片段判断其结局倾向。

该任务的输入是一段 TinyStories 风格的英文故事结尾文本，输出类别为 `positive` 或 `negative`。其中，`positive` 表示故事结局积极，例如问题得到解决、人物获得帮助、冲突缓和、人物最终安全或开心；`negative` 表示故事结局消极，例如问题没有解决、人物受伤、失败、伤心、害怕、孤单，或坏结果持续存在。与普通影评或商品评论情感分类相比，该任务与预训练语料的儿童短故事风格更加一致，因此更适合用来展示本项目中小型 GPT 模型的任务迁移过程。

监督数据由 TinyStories 风格文本构建而来。由于原始故事文本不包含分类标签，本文先抽取候选故事结尾，再依据结局倾向进行人工标注。标注时只依据当前文本中呈现的信息判断：如果困难最终被解决，则标为 `positive`；如果困难没有解决或负面结果仍然存在，则标为 `negative`；结局不清楚或难以判断的候选样本不进入最终训练集。最终主实验构建了 300 条带标签的故事结尾样本，数据格式由文本字段和标签字段组成。其中，文本字段为故事结尾片段，标签字段为人工标注的结局倾向类别。该格式既便于程序读取，也能够直接对应文本分类任务的输入和输出，具体文件路径见附录。

数据读取后，程序复用预训练阶段保存的 `tokenizer` 对文本进行编码，并根据模型上下文长度进行截断和 `padding`，同时生成 `attention_mask` 用于区分真实 `token` 与 `padding token`。数据集采用固定随机种子进行分层划分，训练集、验证集和测试集数量分别为 210、45 和 45。由于测试集中正负类别样本数量并非完全相同，除 `accuracy` 外，本文还报告 `precision`、`recall`、`F1`、`macro F1` 和 `weighted F1`，以更全面地衡量模型在不同类别上的表现。表 4-1 给出了微调数据集划分与标签设置。

表 4-1 微调数据集划分与标签设置

项目	设置	数量	说明
任务名称	故事结局倾向二分类	300	判断故事结尾为积极或消极
标签 0	negative	—	结局消极、问题未解决、人物受伤或难过
标签 1	positive	—	结局积极、问题解决、人物安全或获得帮助
训练集	train	210	用于参数更新
验证集	validation	45	用于选择验证集表现最优的模型
测试集	test	45	仅用于最终效果评估

4.2 微调策略与程序设计

微调模型在预训练 MiniGPT 主体后增加一个线性分类头，而不改变原有 GPT 主体结构。输入文本经过 token embedding、position embedding、Transformer blocks 和 final norm 后，得到每个 token 的隐藏状态。分类任务需要整段文本的表示，因此本文采用 mean pooling 将所有有效 token 的隐藏状态合成为文本级向量，再送入线性分类头，输出 negative 和 positive 两个类别的 logits。训练目标使用交叉熵损失。

在初始化方式上，微调程序加载预训练阶段验证集表现最优的 MiniGPT 参数，并复用预训练阶段的 tokenizer 和词表，从而保证预训练与微调阶段的 token 编码方式一致。参数更新策略上，本文最终采用全参数微调，即 MiniGPT 主体与分类头全部参与更新。这样做的原因是本项目中的 MiniGPT 规模较小，仅包含 2 层 Transformer block，若只训练分类头或最后一层，模型对下游分类任务的适配能力会受到限制。

训练使用 AdamW 优化器，学习率为 5×10^{-5} ，权重衰减为 0.01，batch size 为 8，训练轮数上限为 60。训练过程中根据验证集表现保存最佳模型，并在最终测试时使用该最佳模型，而不是简单使用最后一轮模型。程序记录训练损失、验证损失、

accuracy、分类报告、混淆矩阵以及错误样本，用于后续分析模型在不同类别上的表现。表 4-2 汇总了主实验参数设置。

表 4-2 微调实验参数设置

参数	设置
微调数据	300 条故事结尾标注样本
初始化方式	加载预训练阶段验证集表现最优的 MiniGPT 参数
Tokenizer	复用预训练阶段词表
参数更新方式	全参数微调，MiniGPT 主体与分类头全部参与更新
文本表示方式	mean pooling
最大长度	64 tokens
batch size	8
learning rate	5×10^{-5}
weight decay	0.01
epochs	60
early stopping patience	30
可训练参数量	1,228,418

4.3 微调结果与任务效果分析

最终实验结果由微调程序自动记录，包括训练曲线、测试集预测结果、分类报告和错误样本分析。图 4-1 和图 4-2 分别展示了微调过程中的 loss 与 accuracy 变化。具体图片文件路径见附录。

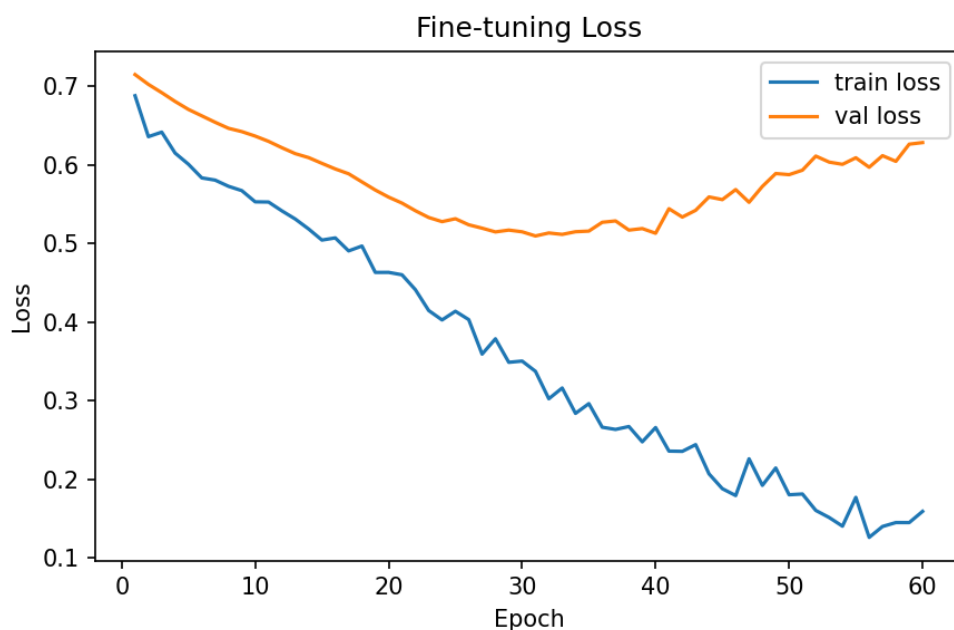


图 4-1 微调训练 loss 曲线

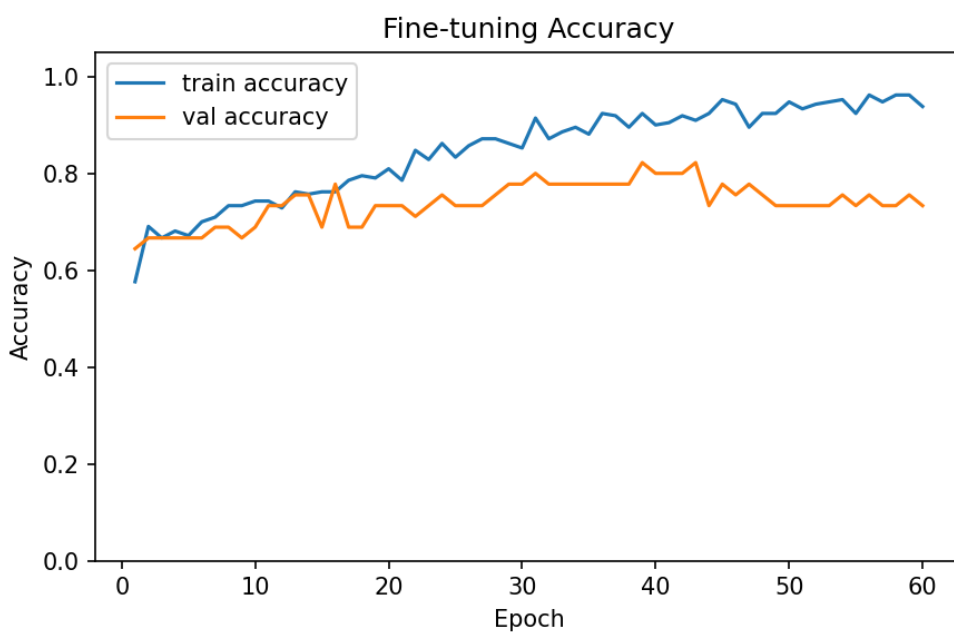


图 4-2 微调 accuracy 曲线

从训练过程看，随着 epoch 增加，训练集 loss 整体下降，训练准确率逐步升高，说明模型能够从监督数据中学习故事结局倾向的分类特征。验证集在第 39 轮达到最佳表现，验证准确率为 82.22%，验证损失为 0.5188。后续训练中，训练集准确率继

续升高，而验证集准确率和验证损失出现波动，说明模型后期存在一定过拟合趋势。因此，最终测试阶段使用第 39 轮对应的验证集最优模型进行评估。

在测试集上，模型准确率为 84.44%，macro F1 为 80.62%，weighted F1 为 83.87%。测试集分类指标如表 4-3 所示。混淆矩阵中，真实 negative 被预测为 positive 的样本有 5 条，真实 positive 被预测为 negative 的样本有 2 条，说明模型对 positive 类识别更稳定，而对部分负向结局，尤其是文本中同时包含“学到教训”等积极表达的样本，仍容易发生误判。

表 4-3 测试集分类指标

类别/指标	Precision	Recall	F1	Support
negative	81.82%	64.29%	72.00%	14
positive	85.29%	93.55%	89.23%	31
macro avg	83.56%	78.92%	80.62%	45
weighted avg	—	—	83.87%	45
overall accuracy	84.44%			
confusion matrix	tn=9, fp=5, fn=2, tp=29			

表 4-4 给出了部分测试样例及预测结果。可以看到，模型能够较好识别“成为朋友”“安全回家”“得到帮助”等积极结局，也能识别人物受伤、结局悲伤等明显负向结局。错误样例主要集中在边界文本上，例如故事中先出现负面事件，但结尾又包含“learned to be careful”一类带有积极含义的表达，模型可能过度关注“学到教训”而忽略前面更关键的负向结果。

表 4-4 测试样例与预测结果

文本摘要	真实标签	预测标签	类别概率	结果
Tom and Lucy became good friends. They played and cleaned together every day.	positive	positive	$P_{pos} = 0.933$	正确
The pup lay in his hands, still and quiet. The story ends sadly.	negative	negative	$P_{neg} = 0.799$	正确
The snake got home safe and sound after being careful in the rain.	positive	positive	$P_{pos} = 0.965$	正确
The animals were sad and missed their friend, but learned to be careful.	negative	positive	$P_{pos} = 0.905$	错误
The mouse was hurt and could not move, but learned to be careful.	negative	positive	$P_{pos} = 0.840$	错误
Timmy and the dog became best friends after going home.	positive	positive	$P_{pos} = 0.993$	正确

与预训练阶段主要用于文本续写不同，微调后的模型能够根据输入故事结尾输出明确的类别标签，说明预训练 MiniGPT 已经能够通过监督数据适配特定分类任务。这一结果表明，即使模型规模较小，只要下游数据与预训练语料在风格上较为一致，预训练模型仍可以通过简单分类头和监督微调获得可观察的任务迁移效果。

4.4 本章小结

本章围绕故事结局倾向二分类任务完成了监督微调实验。实验构建了带标签的故事结尾数据，确定了全参数微调与 mean pooling 的分类策略，并在预训练 MiniGPT 基础上实现了分类微调程序。实验通过 accuracy、F1、混淆矩阵和测试样例展示了模型处理特定任务的效果，测试集准确率达到 84.44%。同时，结果也表明小模型和小规模数据仍存在局限：模型对 positive 类识别更稳定，对包含负面事件但带有积极措辞的边界样本仍容易误判，后续可通过补充负向样本和改进标注一致性进一步提升

泛化能力。

参考文献与开源资源说明

数据集

附录

附录 A：项目文件结构

附录 B：主要运行命令

附录 C：核心代码说明

附录 D：实验结果截图清单